



Конспект по теме «Основные компоненты Kubernetes»

Содержание

Нажмите на название раздела, чтобы перейти к нему

[Основные компоненты Kubernetes](#)

[Взаимодействие компонентов](#)

[Важность понимания архитектуры для установки](#)

[Дополнительные материалы](#)

Основные компоненты Kubernetes

Kubernetes состоит из нескольких ключевых компонентов, которые обеспечивают работу кластера. Их можно разделить на две группы:

- Компоненты Control Plane (мастер-ноды) — управляют кластером.
- Компоненты Worker Nodes (воркер-ноды) — выполняют задачи.

1.1. Компоненты Control Plane

1. kube-apiserver

- Назначение: главный интерфейс для управления кластером. Принимает запросы от пользователей и других компонентов.
- Пример: при выполнении команды `kubectl get pods` запрос отправляется в kube-apiserver.
- Важность: без kube-apiserver кластер продолжит работать, но не получится им управлять

2. etcd

- Назначение: хранит все данные о состоянии кластера (конфигурации, секреты, метаданные).
- Важность: без etcd кластер перестаёт работать.

3. **kube-scheduler**

- Назначение: определяет, на какой ноде запускать поды, учитывая ресурсы и требования.
- Пример: если под требует 2 ГБ ОЗУ, scheduler выбирает ноду с достаточными ресурсами.

4. **kube-controller-manager**

- Назначение: управляет контроллерами (например, ReplicaSet, Deployment), обеспечивая соответствие текущего состояния желаемому.

5. **cloud-controller-manager** (опционально)

- Назначение: интегрирует кластер с облачными провайдерами (например, AWS, Yandex Cloud).

1.2. Компоненты Worker Nodes

1. **kubelet**

- Назначение: Агент на каждой ноде, который запускает и управляет подами.
- Пример: Если мастер отправляет команду на запуск пода, kubelet выполняет её на своей ноде.

2. **kube-proxy**

- Назначение: Обеспечивает сетевое взаимодействие между подами и внешним миром (например, через iptables или IPVS)

3. **Container Runtime** (Docker, containerd, CRI-O)

- Назначение: Запускает контейнеры внутри подов.

1.3. Дополнительные компоненты

- **DNS** — у кластера может быть своя система доменных имен. Она позволяет общаться с подами (Pod) через сервисы (Service), используя доменные имена вместо IP адресов. ClusterDNS — основной компонент. Эта система необходима, так как с ней работают много разных дополнительных модулей.

- **Web UI (Dashboard)** — можно развернуть веб-интерфейс, представляющий дашборд работы кластера. Он не обязателен, но добавляет удобства в восприятии кластера.

Взаимодействие компонентов

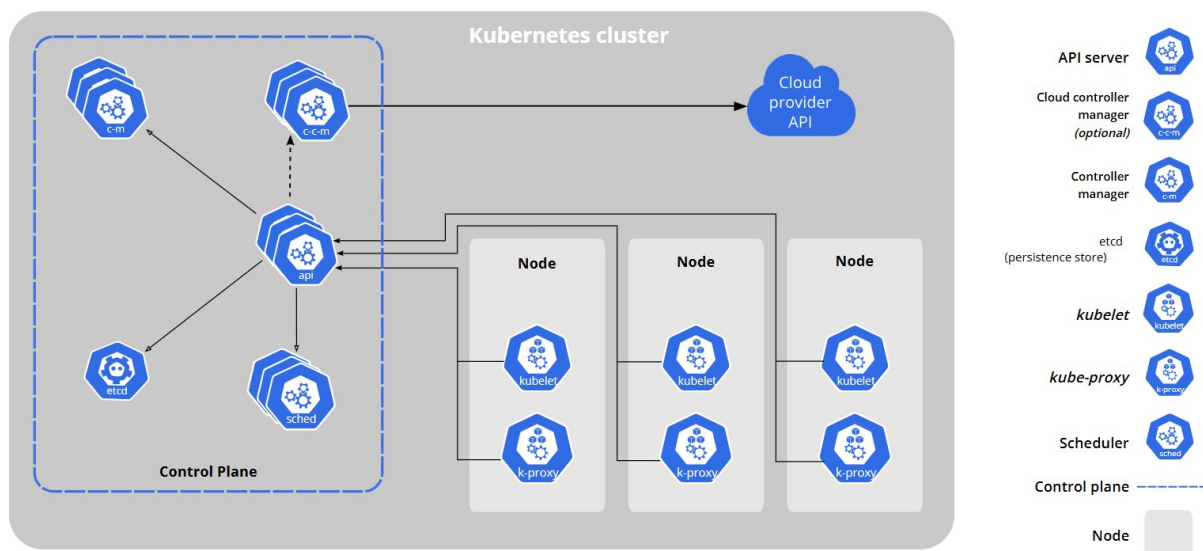
Пример работы кластера:

1. Пользователь отправляет запрос через `kubectl` в **kube-apiserver**.
2. **kube-scheduler** определяет, куда разместить под.
3. **kube-controller-manager** следит за состоянием пода.
4. **kubelet** на ноде запускает контейнер через **Container Runtime**.
5. **kube-proxy** настраивает сетевые правила для доступа к поду.

На примере работы городских служб:

1. Гражданин приходит в мэрию (`kube-apiserver`) и просит построить дом (пользователь запускает под командой `kubectl exec`).
2. Отдел планирования (`scheduler`) выбирает участок.
3. Инспекторы (`controller-manager`) проверяют процесс.
4. Управляющий (`kubelet`) строит дом (под).
5. Дорожные службы (`kube-proxy`) проводят коммуникации.

Схема архитектуры Kubernetes



Источник

Важность понимания архитектуры для установки

1. Выбор инструментов:

- **kubeadm** требует ручной настройки компонентов .
- **Kubespray** автоматизирует установку через Ansible.

2. Отказоустойчивость:

- Для HA (High Availability) нужны несколько мастер-нод с etcd и балансировщиком (например, Keepalived)

3. Проблемы:

- Если kubelet не запущен на ноде, поды не будут работать

Дополнительные материалы

- [Kubernetes The Hard Way](#) — ручная установка компонентов. Автор Kelsey Hightower
- [Официальная документация kubespray](#)

Понимание роли каждого компонента Kubernetes важно для корректной установки и устранения неполадок. Инструменты Kubespray или kubeadm упрощают процесс, при этом знание архитектуры помогает адаптировать кластер под конкретные задачи.